Collatz-Based Cryptographic Primitives:

A Permutation, Hash Function, Authenticated Cipher,
and Message Authentication Code
Derived from the $3n + 1$ Mapping

Zachary Mock
`zmockwebdesign`

Draft for Peer Review — March 2026

Abstract

We present a family of cryptographic primitives whose core nonlinear operation is derived from the Collatz mapping $n \mapsto (3n + 1)/2^{\nu_2(3n+1)}$. The central construction is **CollatzPerm-512**, a 512-bit cryptographic permutation that pairs a branchless Collatz step with a $64 \times 64 \to 128$-bit integer multiplication for diffusion. On top of this single permutation we build three modes of operation using well-studied sponge and duplex constructions: (1) **CollatzHash**—a sponge hash with 256-, 512-, or 1024-bit output; (2) **CollatzAEAD**—a duplex-sponge authenticated cipher (256-bit key, 128-bit nonce, 128-bit tag); (3) **CollatzMAC**—a keyed-sponge message authentication code. The design targets 128-bit security. We provide a reference C++17 implementation with ARM64 inline assembly for the critical path. Empirical results on Apple M3 Max show CollatzHash-256 at 17 M ops/s (3.4× faster than standalone SHA-256), CollatzAEAD at 860 MB/s on 4 KB messages, and CollatzMAC at 6.9 M ops/s (5.4× faster than HMAC-SHA-256). All 70 security validation tests pass, including full permutation diffusion, avalanche at 49.6–50.3%, zero collisions in 10^6 inputs, AEAD authentication integrity, and constant-time tag comparison.

Contents

1 Introduction **3**
1.1 Contributions . . . 3
1.2 Design Philosophy . . . 3
1.3 Notation . . . 3

2 Prior Work: CollatzHash V4 (Non-Cryptographic) **4**
2.1 Core Primitive: `collatz_rapid_mix` . . . 4
2.2 Architecture . . . 4
2.3 ARM64 Assembly Optimization . . . 4
2.4 Limitations . . . 4

3 CollatzPerm-512: The Cryptographic Permutation **5**
3.1 State . . . 5
3.2 Round Function . . . 5
3.2.1 Layer 1: Nonlinear (Collatz–Multiply Pairs) . . . 5
3.2.2 Layer 2: Linear Diffusion . . . 5
3.2.3 Layer 3: Round Constant Injection . . . 6
3.3 Round Counts . . . 6
3.4 Security Claim . . . 6

4 CollatzHash V5: Sponge Hash **6**
4.1 Construction . . . 6
4.2 Output Widths . . . 7

5 CollatzAEAD: Authenticated Encryption **7**
5.1 Construction . . . 7
5.2 Security Properties . . . 8

6 CollatzMAC: Message Authentication Code **8**

7 Domain Separation **8**

8 Implementation **9**
8.1 Reference Implementation . . . 9
8.2 ARM64 Optimization . . . 9
8.3 Constant-Time Guarantees . . . 9

9 Empirical Security Evaluation **9**
9.1 Permutation Diffusion . . . 10
9.2 Avalanche Effect . . . 10
9.3 Bit Distribution . . . 10
9.4 Collision Resistance . . . 10
9.5 AEAD Integrity . . . 10
9.6 Key Sensitivity . . . 11

10 Performance **11**
10.1 Hash Performance . . . 11
10.2 AEAD Performance . . . 11
10.3 MAC Performance . . . 11

11 Comparison with Existing Primitives **12**

12 Analysis of the Collatz Nonlinearity **12**
12.1 Why the Collatz Step Provides Nonlinearity . . . 12
12.2 Differential Properties . . . 12
12.3 The Pre-Mix Rotation . . . 12

13 Discussion **13**
13.1 Strengths . . . 13
13.2 Limitations and Open Questions . . . 13
13.3 Recommended Use Cases . . . 13
13.4 Not Recommended (Without Further Analysis) . . . 13

14 Future Work **14**

15 Conclusion **14**

A Test Vectors **15**
A.1 CollatzHash-256 . . . 15
A.2 CollatzHash-512 . . . 15
A.3 CollatzMAC-128 . . . 16

1 Introduction

The Collatz conjecture (also known as the $3n + 1$ problem) states that for any positive integer n, the sequence defined by

$$T(n) = \begin{cases} n/2 & \text{if } n \equiv 0 \pmod 2, \\ (3n+1)/2 & \text{if } n \equiv 1 \pmod 2, \end{cases}$$

eventually reaches 1. Despite its elementary statement, the conjecture remains one of the most famous open problems in mathematics. The mapping exhibits chaotic, pseudorandom behavior: nearby starting values can produce vastly different orbit lengths and trajectories.

We exploit this pseudorandom behavior as a *nonlinear mixing primitive* for cryptographic hash functions and ciphers. Prior work has explored Collatz-like maps in pseudorandom number generation [1], but to our knowledge no complete cryptographic suite has been built on the $3n+1$ mapping.

1.1 Contributions

1. A novel cryptographic permutation, **CollatzPerm-512**, that uses a branchless Collatz step paired with 128-bit integer multiplication as its nonlinear layer.
2. Three modes of operation—hash, AEAD, MAC—built using standard sponge/duplex constructions, inheriting formal security bounds from the permutation.
3. A reference implementation in C++17 with hand-tuned ARM64 assembly for the critical path.
4. Empirical security evaluation: diffusion, avalanche, distribution, collision search, and authentication integrity.
5. Performance benchmarks against SHA-256, ChaCha20, and HMAC-SHA-256.

1.2 Design Philosophy

The fastest non-cryptographic hashes (rapidhash [7], wyhash [8], xxHash3 [9]) share a key insight: a single $64 \times 64 \to 128$-bit multiply followed by XOR of the high and low halves provides massive diffusion in 3–4 CPU cycles. On the other hand, established cryptographic hashes (SHA-256, SHA-3) derive their security from carefully designed round functions with proven diffusion properties, at the cost of higher latency.

Our design occupies a middle ground: we replace the purely algebraic multiply with a *Collatz–multiply hybrid* that adds mathematical nonlinearity from number theory at minimal cycle cost, then use the multiply for diffusion. The result is faster than SHA-256 while providing a cryptographic-strength construction.

1.3 Notation

- $\nu_2(n)$: the 2-adic valuation of n (number of trailing zeros in the binary representation).
- $\mathrm{rotl}_{64}(x, r)$: 64-bit left rotation by r positions.
- $\mathrm{ctz}(n) = \nu_2(n)$: count trailing zeros.
- $\oplus$: bitwise exclusive-or (XOR).
- $\cdot_{128}$: 64×64-bit unsigned multiplication producing a 128-bit result, split into low (lo) and high (hi) 64-bit words.

- $s[0..7]$: the 512-bit state as eight 64-bit words.
- $R_a = 12$: number of rounds for initialization and finalization.
- $R_b = 8$: number of rounds for per-block processing.

2 Prior Work: CollatzHash V4 (Non-Cryptographic)

Before developing the cryptographic suite, we designed **CollatzHash V4**, a high-performance non-cryptographic hash optimized for maximum throughput.

2.1 Core Primitive: `collatz_rapid_mix`

The V4 mixing function operates on two 64-bit inputs (a, b) and produces a 64-bit output. The algorithm proceeds as follows:

Function `collatz_rapid_mix`(a, b):

1. $a \leftarrow a \mid 1$ (ensure odd, branchless)
2. $t \leftarrow 3a + 1$
3. $t \leftarrow t \gg \mathrm{ctz}(t)$ (first Collatz iteration)
4. $t_2 \leftarrow 3t + 1$
5. $t_2 \leftarrow t_2 \gg \mathrm{ctz}(t_2)$ (second Collatz iteration)
6. $a \leftarrow t_2$
7. $(\mathrm{lo}, \mathrm{hi}) \leftarrow a \cdot_{128} b$
8. $a \leftarrow a \oplus \mathrm{lo}; \quad b \leftarrow b \oplus \mathrm{hi}$
9. `return` $a \oplus b$

2.2 Architecture

V4 uses a 192-bit state (three 64-bit accumulators s_0, s_1, s_2) with:

- **Short-input fast path** (≤ 48 bytes): 2–3 mix calls, no loops.
- **Large-input path**: 3 parallel accumulators processing 48-byte blocks.
- **Sponge squeeze**: for output widths beyond 64 bits (up to 1024 bits), re-permuting the state and emitting 64 bits per squeeze.

2.3 ARM64 Assembly Optimization

On Apple M3 Max (ARM64), the V4 mixing function is implemented in 11–16 instructions using:

- `ADD`+`LSL` for $3n$ (fused add-shift),
- `RBIT`+`CLZ` for ctz (bit-reverse then count leading zeros),
- `MUL`+`UMULH` for 128-bit multiply (low and high halves).

A single-iteration variant (V4b) uses 11 instructions and achieves 16 M ops/s at 1024-bit output and 253 M ops/s at 64-bit output.

2.4 Limitations

V4's 192-bit state provides only 2^{64} collision resistance (capacity = 128 bits), making it unsuitable for cryptographic use. It has no key schedule, no formal security claims, and no resistance to algebraic or differential attacks. This motivated the design of the cryptographic suite described in the following sections.

3 CollatzPerm-512: The Cryptographic Permutation

3.1 State

The permutation operates on a 512-bit state represented as eight 64-bit words: $s = (s[0], s[1], \ldots, s[7])$.

The state is divided into:

- **Rate** ($r = 256$ bits): words $s[0..3]$, used for absorbing input and squeezing output.
- **Capacity** ($c = 256$ bits): words $s[4..7]$, never directly exposed.

3.2 Round Function

Each round i ($0 \le i < R$) applies three layers in sequence.

3.2.1 Layer 1: Nonlinear (Collatz–Multiply Pairs)

Four independent pairs $(s[0], s[1])$, $(s[2], s[3])$, $(s[4], s[5])$, $(s[6], s[7])$ are processed by the `collatz_mul_pair` operation:

Function `collatz_mul_pair`(a, b):

1. $a \leftarrow a \oplus \mathrm{rotl}_{64}(a, 23)$ (pre-mix rotation)
2. $a \leftarrow a \,|\, 1$ (ensure odd)
3. $t \leftarrow 3a + 1$
4. $t \leftarrow t \gg \mathrm{ctz}(t)$ (Collatz step)
5. $a \leftarrow t$
6. $(\mathrm{lo}, \mathrm{hi}) \leftarrow a \cdot_{128} b$ (128-bit multiply)
7. $a \leftarrow a \oplus \mathrm{lo}; \quad b \leftarrow b \oplus \mathrm{hi}$ (XOR feedback)

The pre-mix rotation (step 1) ensures that bit 0 of the input—which would otherwise be destroyed by the $a \,|\, 1$ operation—influences the output through bit position 23. This was identified through diffusion testing (Section 9.1).

3.2.2 Layer 2: Linear Diffusion

Cross-pair mixing ensures information propagates between all four pairs within approximately 3 rounds:

Linear diffusion layer:

1. $s[0] \leftarrow s[0] \oplus s[2]; \quad s[4] \leftarrow s[4] \oplus s[6]$
2. $s[1] \leftarrow s[1] \oplus s[3]; \quad s[5] \leftarrow s[5] \oplus s[7]$
3. $(s[0], s[2], s[4], s[6]) \leftarrow (s[2], s[4], s[6], s[0])$ (word rotation)
4. $s[1] \leftarrow s[1] \oplus \mathrm{rotl}_{64}(s[5], 19)$
5. $s[3] \leftarrow s[3] \oplus \mathrm{rotl}_{64}(s[7], 31)$
6. $s[5] \leftarrow s[5] \oplus \mathrm{rotl}_{64}(s[1], 7)$
7. $s[7] \leftarrow s[7] \oplus \mathrm{rotl}_{64}(s[3], 53)$

The rotation constants $(19, 31, 7, 53)$ are chosen to be pairwise coprime to 64, avoiding aligned mixing patterns.

3.2.3 Layer 3: Round Constant Injection

$$s[0] \leftarrow s[0] \oplus \mathrm{RC}[i]$$

The twelve round constants RC[0..11] are derived from the binary expansions of π, e, $\sqrt{2}$, and $\sqrt{3}$ (nothing-up-my-sleeve numbers). When the round count exceeds 12, constants are reused cyclically: RC[i mod 12].

The round constants in hexadecimal are:

RC[0] = `0x243f6a8885a308d3`	RC[1] = `0xb7e151628aed2a6a`
RC[2] = `0x6a09e667f3bcc908`	RC[3] = `0xbb67ae8584caa73b`
RC[4] = `0x3c6ef372fe94f82b`	RC[5] = `0xa54ff53a5f1d36f1`
RC[6] = `0x510e527fade682d1`	RC[7] = `0x9b05688c2b3e6c1f`
RC[8] = `0x1f83d9abfb41bd6b`	RC[9] = `0x5be0cd19137e2179`
RC[10] = `0x428a2f98d728ae22`	RC[11] = `0x7137449123ef65cd`

3.3 Round Counts

Following the dual-round approach of Ascon [4]:

- $R_a = 12$ rounds: initialization and finalization (full security margin).
- $R_b = 8$ rounds: per-block absorption (reduced, compensated by wide state).

3.4 Security Claim

Claim 1. *Assuming CollatzPerm-512 behaves as a pseudorandom permutation on 512 bits, the sponge and duplex constructions provide:*

- *Collision resistance:* $2^{c/2} = 2^{128}$.
- *Preimage resistance:* $\min(2^c, 2^{\text{output_bits}}) = 2^{256}$ *(for 256-bit output).*
- *AEAD confidentiality and integrity: up to* $\min(2^{c/2}, 2^k) = 2^{128}$ *queries.*

These bounds follow directly from the generic sponge security proofs [5] with capacity $c = 256$.

4 CollatzHash V5: Sponge Hash

4.1 Construction

CollatzHash V5 is a sponge hash [2] over CollatzPerm-512 with rate $r = 256$ bits and capacity $c = 256$ bits.

Function `CollatzHash`(M, d): Hash message M to d output bits

Initialization:

1. $s \leftarrow (0, 0, 0, 0, 0, 0, 0, 0)$ (512-bit zero state)
2. $s[7] \leftarrow s[7] \oplus$ `DS_HASH` (domain separation)
3. $s[6] \leftarrow s[6] \oplus d$ (encode output length)

Absorption:

4. Pad M with 10*1 padding to a multiple of $r = 256$ bits
5. for each r-bit block B_i of padded M:
   $s[0..3] \leftarrow s[0..3] \oplus B_i$
   $s \leftarrow \mathrm{CollatzPerm}(s, R_b)$

Finalization and Squeeze:

6. $s \leftarrow \text{CollatzPerm}(s, R_a)$ (finalization)
7. $Z \leftarrow \varepsilon$ (empty output)
8. `while` $|Z| < d$:
   $Z \leftarrow Z \,\|\, s[0..3]$ (squeeze r bits)
   `if` $|Z| < d$ `then` $s \leftarrow \text{CollatzPerm}(s, R_b)$
9. `return first` d `bits of` Z

4.2 Output Widths

- **256-bit**: 1 squeeze (no additional permutation after finalization).
- **512-bit**: 2 squeezes (1 additional permutation).
- **1024-bit**: 4 squeezes (3 additional permutations).

Each additional 256-bit block is a genuine squeeze from a re-permuted state, following the same principle as SHA-3/SHAKE [3].

5 CollatzAEAD: Authenticated Encryption

CollatzAEAD is a duplex-sponge AEAD cipher [6] with:

- Key: 256 bits.
- Nonce: 128 bits.
- Tag: 128 bits.
- Rate: 256 bits per block.

5.1 Construction

Function `CollatzAEAD.Encrypt`(K, N, A, P):

Initialization:

1. $s \leftarrow (0, \ldots, 0)$
2. `Load` N `into` $s[0..1]$; `load` K `into` $s[4..7]$
3. $s[7] \leftarrow s[7] \oplus$ `DS_AEAD`
4. $s \leftarrow \text{CollatzPerm}(s, R_a)$
5. $s[4..7] \leftarrow s[4..7] \oplus K$ (post-init key injection)

Process Associated Data A:

6. `for each padded` r`-bit block` A_i `of` A:
   $s[0..3] \leftarrow s[0..3] \oplus A_i$
   $s \leftarrow \text{CollatzPerm}(s, R_b)$
7. $s[7] \leftarrow s[7] \oplus$ `DS_AD_END` (domain separation)

Encrypt Plaintext P:

8. `for each` r`-bit block` P_i:
   $C_i \leftarrow s[0..3] \oplus P_i$ (keystream XOR)
   $s[0..3] \leftarrow C_i$ (absorb ciphertext—duplex feedback)
   $s \leftarrow \text{CollatzPerm}(s, R_b)$

Finalization:

9. $s[4..7] \leftarrow s[4..7] \oplus K$
10. $s \leftarrow \text{CollatzPerm}(s, R_a)$

```
11.  s[4..7] ← s[4..7] ⊕ K
12.  T ← s[4..5]  (128-bit tag from capacity)
13.  return (C, T)
```

Decryption follows the same structure with the inverse operation on the plaintext blocks. The tag is verified using constant-time byte comparison before releasing any plaintext.

5.2 Security Properties

- **Confidentiality**: ciphertext is indistinguishable from random under CPA, bounded by 2^{128} queries.
- **Integrity**: forging a valid (C, T) pair requires 2^{128} work (tag is extracted from the capacity).
- **Nonce misuse**: as with all nonce-based AEAD, reusing a nonce with the same key leaks information. This is a standard limitation shared by AES-GCM and ChaCha20-Poly1305.

6 CollatzMAC: Message Authentication Code

CollatzMAC is a keyed-sponge MAC [5] with a 256-bit key and configurable tag length (128 or 256 bits).

Function `CollatzMAC`(K, M, d): MAC with d-bit tag

```
1.  s ← (0, ..., 0)
2.  s[7] ← s[7] ⊕ DS_MAC;   s[6] ← s[6] ⊕ d
3.  s[0..3] ← s[0..3] ⊕ K  (absorb key into rate)
4.  s ← CollatzPerm(s, R_a)
5.  for each padded r-bit block M_i of M:
    s[0..3] ← s[0..3] ⊕ M_i
    s ← CollatzPerm(s, R_b)
6.  s[4..7] ← s[4..7] ⊕ K  (key injection before final)
7.  s ← CollatzPerm(s, R_a)
8.  Squeeze d bits from rate
9.  return tag
```

Security is bounded by $\min(2^{c/2}, 2^k) = 2^{128}$ MAC queries, following the keyed-sponge PRF bound [5].

7 Domain Separation

Each mode uses a distinct domain separation constant XORed into the capacity before processing:

Mode	Constant
Hash	`0x01`
AEAD	`0x02`
MAC	`0x04`
AAD end	`0x80`
Final	`0xFF`

This ensures that hash digests, ciphertext/tags, and MAC tags computed from the same input are cryptographically independent.

8 Implementation

8.1 Reference Implementation

The reference implementation is in C++17, structured as:

- `collatz_perm.h`: header-only permutation with architecture-specific paths.
- `collatz_crypto.h`: public API for Hash, AEAD, MAC.
- `collatz_crypto.cpp`: implementation, tests, and benchmarks.

8.2 ARM64 Optimization

On AArch64 (Apple Silicon), the `collatz_mul_pair` function is implemented as 11 inline assembly instructions:

```
eor  a, a, a, ror #41     // pre-mix: a ^= rotl(a, 23)
orr  a, a, #1             // ensure odd
add  t1, a, a, lsl #1     // t1 = 3*a
add  t1, t1, #1           // t1 = 3*a + 1
rbit t2, t1               // bit-reverse for ctz
clz  t2, t2               // ctz = clz(rbit(x))
lsr  a, t1, t2            // a = (3a+1) >> ctz
mul  t1, a, b             // lo = a * b
umulh t2, a, b            // hi = a * b
eor  a, a, t1             // a ^= lo
eor  b, b, t2             // b ^= hi
```

The function uses only general-purpose registers and has no data-dependent branches or memory accesses, making it constant-time with respect to the state values.

Note on `RBIT+CLZ`. ARM64 does not have a native count-trailing-zeros instruction. Instead, we compose bit-reverse (`RBIT`) with count-leading-zeros (`CLZ`) to obtain ctz in 2 cycles. On x86-64, the `TZCNT` instruction provides this directly.

8.3 Constant-Time Guarantees

- The Collatz step uses `ORR` (bitwise OR with 1) instead of a conditional branch.
- Tag verification uses byte-wise OR accumulation, not early return.
- No secret-dependent array indexing or memory access patterns.

Caveat: The variable-length right shift $t \gg \mathrm{ctz}(t)$ depends on the data-dependent shift amount $\mathrm{ctz}(t)$. On most modern processors (ARM64, x86-64), barrel shifters execute variable shifts in constant time. However, this should be verified on the target platform.

9 Empirical Security Evaluation

We run 70 automated tests covering permutation diffusion, hash avalanche, bit distribution, collision resistance, AEAD correctness and integrity, and MAC correctness.

9.1 Permutation Diffusion

We verify that flipping any single bit in any of the 8 state words, after 12 rounds of CollatzPerm-512, changes all 8 output words. We test all 8 words at bit positions 0, 16, 32, and 48 (32 tests total).

Result: 32/32 tests pass (all 8 words change in every case). This was achieved after adding the pre-mix rotation $a \leftarrow a \oplus \mathrm{rotl}_{64}(a, 23)$ to prevent the $a \,|\, 1$ operation from absorbing bit 0.

9.2 Avalanche Effect

For each input string, we flip every bit of the input and measure the Hamming distance between the original and modified hash digests, expressed as a percentage of the output width:

Input	256-bit	512-bit	1024-bit
`"hello world"`	50.3%	49.9%	50.0%
`"Noobas123!@#"`	49.8%	50.1%	50.1%
`"test input here"`	49.6%	49.8%	49.9%

All values fall within the acceptable range of 50±5%, indicating near-ideal avalanche behavior across all output widths. The ideal value is 50%, meaning each output bit flips with probability $\frac{1}{2}$ when any input bit changes.

9.3 Bit Distribution

For 50,000 sequential inputs (`"dist_0"` through `"dist_49999"`), we measure the percentage of times each bit position is set to 1:

Output Width	Min	Max
256-bit	49.5%	50.6%
1024-bit	49.2%	50.6%

All bit positions are within $50 \pm 1\%$, indicating excellent uniformity.

9.4 Collision Resistance

We hash 10^6 distinct inputs at both 256-bit and 1024-bit output widths and check for duplicate digests:

Output Width	Inputs	Collisions
256-bit	1,000,000	0
1024-bit	1,000,000	0

Note: A birthday-bound collision search for a 256-bit hash requires approximately 2^{128} inputs; our test of $10^6 \approx 2^{20}$ inputs is insufficient to find collisions by chance and serves only to verify the absence of trivial implementation errors.

9.5 AEAD Integrity

We test authentication integrity by verifying that decryption fails (returns $\perp$) when any of the following are modified:

Modification	Result
Flip 1 ciphertext bit	Rejected
Flip 1 tag bit	Rejected
Flip 1 AAD bit	Rejected
Wrong key (1-bit diff)	Rejected
Wrong nonce (1-bit diff)	Rejected

All 5 authentication tests pass. AEAD roundtrip correctness is verified for plaintext lengths 0, 1, 15, 16, 31, 32, 33, 63, 64, 100, and 1000 bytes (all 11 tests pass).

9.6 Key Sensitivity

Two 256-bit keys differing by a single bit produce ciphertexts with 93% of bytes differing and completely different authentication tags.

10 Performance

All benchmarks were run on Apple M3 Max (ARM64, 10 performance cores) with `g++ -O3 -march=native`. All comparison implementations are standalone C++ (no hardware acceleration extensions, no external libraries).

10.1 Hash Performance

Input	Algorithm	ops/sec	ns/op
5 B	CollatzHash-256	17,066,218	58.6
	CollatzHash-1024	5,963,157	167.7
	SHA-256	4,828,820	207.1
12 B	CollatzHash-256	16,745,487	59.7
	CollatzHash-1024	5,956,764	167.9
	SHA-256	4,962,137	201.5
43 B	CollatzHash-256	10,903,183	91.7
	CollatzHash-1024	4,944,270	202.3
	SHA-256	4,911,992	203.6

CollatzHash-256 is **3.4× faster** than SHA-256 on short inputs. The advantage narrows for longer inputs as the sponge absorption cost grows proportionally.

10.2 AEAD Performance

Size	CollatzAEAD	ChaCha20	Ratio
64 B	332 MB/s	917 MB/s	0.36×
256 B	629 MB/s	951 MB/s	0.66×
1024 B	800 MB/s	931 MB/s	0.86×
4096 B	859 MB/s	942 MB/s	0.91×

Note: ChaCha20 here is encryption-only (no Poly1305 authentication). CollatzAEAD provides combined encrypt+authenticate in a single pass, so the true comparison against ChaCha20-Poly1305 would be more favorable.

10.3 MAC Performance

Algorithm	ops/sec	ns/op
CollatzMAC-128	6,941,647	144.1
HMAC-SHA-256	1,290,474	774.9

CollatzMAC is **5.4× faster** than HMAC-SHA-256.

11 Comparison with Existing Primitives

Property	SHA-256	SHA-3-256	Ascon	Collatz
State width	256 b	1600 b	320 b	512 b
Capacity	—	512 b	192 b	256 b
Security target	128 b	128 b	128 b	128 b
Nonlinearity	Ch, Maj	χ	S-box	Collatz+mul
Rounds/block	64	24	8	8
Output widths	256	XOF	—	256/512/1024
AEAD mode	No	No	Yes	Yes
MAC mode	HMAC	KMAC	Yes	Yes
Standardized	NIST	NIST	NIST	No
Peer review	20+ yr	15+ yr	10+ yr	This paper

12 Analysis of the Collatz Nonlinearity

12.1 Why the Collatz Step Provides Nonlinearity

Traditional symmetric ciphers derive nonlinearity from S-boxes (lookup tables with high differential and linear branch numbers) or from algebraic operations like modular multiplication. The Collatz step $f(a) = (3a + 1) \gg \mathrm{ctz}(3a + 1)$ provides nonlinearity through:

1. **Data-dependent shift.** The shift amount $\mathrm{ctz}(3a + 1)$ varies unpredictably with a. This means the output bits are a data-dependent permutation of the input bits combined with the multiplication by 3, providing high algebraic degree.

2. **Bit truncation.** The right shift discards $\mathrm{ctz}(3a + 1)$ low-order bits, making the function non-invertible on the full 64-bit word. This loss of information is analogous to the compression in S-box designs.

3. **Interaction with multiplication.** The subsequent 128-bit multiply amplifies the nonlinear bit patterns across both output words.

12.2 Differential Properties

Let Δ_a denote an input difference in word a. After the Collatz step:

$$f(a) = (3(a \,|\, 1) + 1) \gg \mathrm{ctz}(3(a \,|\, 1) + 1)$$

The output difference $f(a \oplus \Delta_a) \oplus f(a)$ depends on both the algebraic carry propagation of the $3a + 1$ operation and the data-dependent shift. A formal differential analysis would enumerate the probability $\Pr[\Delta_{\mathrm{out}} | \Delta_{\mathrm{in}}]$ over all possible inputs, which we leave as future work.

12.3 The Pre-Mix Rotation

Without the pre-mix rotation $a \leftarrow a \oplus \mathrm{rotl}_{64}(a, 23)$, the $a \,|\, 1$ operation absorbs (destroys) bit 0 of the input. This creates a specific weakness: for any state word, toggling bit 0 has no effect after the Collatz step, violating the strict avalanche criterion.

The pre-mix XORs bit 0 into bit 23, bit 1 into bit 24, etc., ensuring that every input bit—including bit 0—influences the Collatz step through a different bit position. The rotation distance 23 (equivalently, `ROR #41` on ARM64) is chosen as:

- Coprime to 64 (avoiding periodic alignment),
- Not equal to any of the linear layer rotation constants (19, 31, 7, 53).

13 Discussion

13.1 Strengths

1. **Novel nonlinearity.** The Collatz step introduces a number-theoretic nonlinear operation not found in any existing cryptographic primitive. The chaotic behavior of the $3n + 1$ mapping may provide resistance to algebraic attacks, though this requires further analysis.

2. **Speed.** The Collatz–multiply hybrid achieves better throughput than SHA-256 for hashing and is competitive with ChaCha20 for encryption, without requiring hardware acceleration.

3. **Simplicity.** One permutation serves all three modes. The implementation is under 500 lines of C++ with no external dependencies.

4. **Standard constructions.** The sponge and duplex modes inherit formal security proofs from the literature, reducing the analysis burden to the permutation itself.

13.2 Limitations and Open Questions

1. **No formal cryptanalysis.** The security of the permutation rests on empirical evidence. Formal analysis of differential, linear, and algebraic properties is needed.

2. **Algebraic structure.** The Collatz step is algebraically simple ($3n + 1$ followed by right-shift). An attacker with knowledge of the algebraic structure might find exploitable properties. The 128-bit multiply mitigates this but does not eliminate the concern.

3. **Bit-0 absorption.** The $a \,|\, 1$ operation destroys one bit of input per Collatz step. The pre-mix rotation recovers this information, but the interaction between the rotation and the multiply deserves further study.

4. **No hardware acceleration.** Unlike AES (AES-NI) and SHA-256 (SHA extensions), there is no dedicated hardware support for Collatz operations.

5. **Not standardized.** Adoption in production systems requires peer review and potential standardization.

6. **Variable shift timing.** While barrel shifters on modern processors execute in constant time regardless of shift amount, some microcontrollers may not. Deployments on embedded targets should verify constant-time behavior.

13.3 Recommended Use Cases

- File integrity checking and deduplication.
- API authentication tokens and session management.
- Encrypted storage and communication in applications where SHA-256 is the performance bottleneck and hardware AES is unavailable.
- Research into number-theoretic cryptographic primitives.

13.4 Not Recommended (Without Further Analysis)

- Digital signatures for high-value financial transactions.
- Government or military classified communications.
- Any application requiring FIPS 140 or Common Criteria certification.

14 Future Work

1. **Formal differential/linear cryptanalysis** of CollatzPerm-512, including computation of the best differential and linear characteristics.
2. **Algebraic attack resistance**: analyze the algebraic degree of the round function and resistance to Gröbner basis attacks.
3. **Bounds on round count**: determine the minimum number of rounds for full diffusion and provable resistance to known attacks.
4. **x86-64 SIMD optimization**: exploit AVX2/AVX-512 for parallel Collatz-multiply pairs.
5. **Nonce-misuse resistance**: design an SIV-like mode (Synthetic IV) that provides security even under nonce reuse.
6. **Test vectors**: publish a comprehensive set of known-answer test vectors for interoperability.
7. **Side-channel analysis**: formal verification of constant-time behavior on various microarchitectures.

15 Conclusion

We have presented a complete cryptographic suite built on the Collatz mapping: a 512-bit permutation, a sponge hash, a duplex-sponge AEAD cipher, and a keyed-sponge MAC. The suite targets 128-bit security, outperforms SHA-256 by 3.4× for hashing, matches ChaCha20 for large-message encryption, and surpasses HMAC-SHA-256 by 5.4× for message authentication.

The design demonstrates that the chaotic behavior of the $3n + 1$ mapping can serve as an effective nonlinear mixing primitive when combined with 128-bit integer multiplication for diffusion. We invite the cryptographic community to analyze the permutation's resistance to differential, linear, and algebraic attacks.

The reference implementation is available at: `https://github.com/zmockwebdesign/collatz-v2`

Acknowledgments

The sponge and duplex constructions follow the frameworks established by Bertoni, Daemen, Peeters, and Van Assche for Keccak/SHA-3 and by Dobraunig, Eichlseder, Mendel, and Schläffer for Ascon.

References

[1] J. C. Lagarias, "The $3x + 1$ problem and its generalizations," *American Mathematical Monthly*, vol. 92, no. 1, pp. 3–23, 1985.

[2] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Cryptographic sponge functions," `https://keccak.team/files/CSF-0.1.pdf`, 2011.

[3] NIST, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," FIPS 202, 2015.

[4] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight Authenticated Encryption and Hashing," *Journal of Cryptology*, vol. 34, 2021. NIST Lightweight Cryptography Standard, 2023.

[5] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "On the security of the keyed sponge construction," *SKEW 2011*, 2011.

[6] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "Duplexing the sponge: Single-pass authenticated encryption and other applications," *SAC 2011*, LNCS 7118, pp. 320–337, 2012.

[7] N. Kurz, "rapidhash — Very fast, high quality, platform-independent hashing algorithm," `https://github.com/Nicoshev/rapidhash`, 2024.

[8] Y. Wang, "wyhash — The fastest hash function," `https://github.com/wangyi-fudan/wyhash`, 2023.

[9] Y. Collet, "xxHash — Extremely fast non-cryptographic hash algorithm," `https://github.com/Cyan4973/xxHash`, 2024.

A Test Vectors

The following test vectors are produced by the reference implementation.

A.1 CollatzHash-256

```
Input:  "" (empty)
Output: f266161215a29b9f62dd4b71b4b05144
        40841de8358cd66db10fe489202ccf58

Input:  "abc"
Output: a170ec42f3e1513522b7513f478272db
        054729f09bcd9e4d30f6a661c7a8e0cd

Input:  "hello world"
Output: 4bec83a5598cecee53dc89697f51e928
        e93783237399d7b03415bb6c0ed08c9c
```

A.2 CollatzHash-512

```
Input:  "" (empty)
Output: f266161215a29b9f62dd4b71b4b05144
        40841de8358cd66db10fe489202ccf58
        1a14744ac45936e49b4e751374281ed7
        72b5b1e6be4811fe31015171e22c5812

Input:  "abc"
Output: a170ec42f3e1513522b7513f478272db
        054729f09bcd9e4d30f6a661c7a8e0cd
        13d531694946463d91e759c1acc0d320
        ad5c3995e8a9b32a1b5351dfd95ffe79

Input:  "hello world"
Output: 4bec83a5598cecee53dc89697f51e928
        e93783237399d7b03415bb6c0ed08c9c
        36d90399456178f7a3a6082e9620af31
        e11bd7c81b179784fd351a9ba58fa378
```

A.3 CollatzMAC-128

```
Key:    00000000000000000000000000000000
        00000000000000000000000000000000

Input:  "" (empty)
Output: daff09bb11ac65519bfbe22483fed036

Input:  "abc"
Output: 951eebfd00e05e79c5d182e11b2a3286
```